Notes

Complete Pandas Syllabus — Everything at a Glance (Data Science Edition)

Think of Pandas as a **data manipulation library** built on top of NumPy.

- **NumPy → Numerical Computation**
- **Pandas → Data Analysis & Data Handling**

---

1. Introduction to Pandas

What is Pandas?

Pandas is a Python library used for:

- Data Cleaning
- Data Analysis
- Data Manipulation
- Data Visualization
- Handling CSV, Excel, JSON files

```python
import pandas as pd
```

Pandas vs NumPy

Feature	NumPy	Pandas
Data Type	Arrays	Series, DataFrame
Labels	No	Yes
Missing Values	Limited	Excellent
Data Analysis	Basic	Advanced
CSV/Excel Support	No	Yes

Core Data Structures

Series (1-D)

```
Python
s = pd.Series([10,20,30])
```

Output:

```
Python
0    10
1    20
2    30
```

DataFrame (2-D)

```
Python
df = pd.DataFrame({
    "Name":["Krishna","Rahul"],
    "Age":[20,21]
})
```

Output:

Name	Age
Krishna	20
Rahul	21

2. Pandas Data Structures

Creating Series

```
Python
pd.Series([1,2,3])
```

Custom Index

```
Python
pd.Series([10,20], index=['A','B'])
```

Series Indexing

```
Python
s['A']
```

Series Slicing

```
Python
```

```
s[0:2]
```

Series Operations

```
Python
s + 10
s * 2
s.mean()
```

Creating DataFrames

Dictionary

```
Python
pd.DataFrame({
    "Name":["A","B"],
    "Marks":[90,80]
})
```

List of Lists

```
Python
pd.DataFrame([
    ["A",90],
    ["B",80]
], columns=["Name","Marks"])
```

DataFrame Attributes

Shape

```
Python
df.shape
```

(rows, columns)

Size

```
Python
df.size
```

Total elements

Columns

```
Python
df.columns
```

Index

```python
df.index
```

Dtypes

```python
df.dtypes
```

Values

```python
df.values
```

Returns NumPy Array

---

3. Reading and Writing Data

Reading

CSV

```python
df = pd.read_csv("data.csv")
```

Excel

```python
df = pd.read_excel("data.xlsx")
```

JSON

```python
df = pd.read_json("data.json")
```

HTML Tables

```python
df = pd.read_html(url)
```

Writing

CSV

```python
df.to_csv("output.csv")
```

Excel

```python
df.to_excel("output.xlsx")
```

JSON

```python
df.to_json("output.json")
```

4. Data Inspection & Exploration

First Rows

```python
df.head()
```

Last Rows

```python
df.tail()
```

Random Sample

```python
df.sample(5)
```

Information

```python
df.info()
```

Statistics

```python
df.describe()
```

Frequency Count

```python
df["Gender"].value_counts()
```

Unique Values

```python
df["City"].unique()
```

Count Unique

```python
df["City"].nunique()
```

5. Indexing and Selection

Single Column

```python
df["Name"]
```

Multiple Columns

```python
df[["Name","Age"]]
```

loc (Label Based)

```python
df.loc[0]
```

```python
df.loc[0:3]
```

iloc (Position Based)

```python
df.iloc[0]
```

```python
df.iloc[0:3]
```

Conditional Filtering

```python
df[df["Age"] > 20]
```

6. Data Cleaning

Missing Values

Detect

```python
df.isnull()
```

```python
df.notnull()
```

Remove

```python
df.dropna()
```

Fill

```python
df.fillna(0)
```

```python
df.fillna(df.mean())
```

Duplicate Handling

Check

```python
df.duplicated()
```

Remove

```python
df.drop_duplicates()
```

Rename Columns

```python
df.rename(columns={"Name":"Student"})
```

Replace Values

```python
df.replace("M","Male")
```

Type Conversion

```python
df["Age"] = df["Age"].astype(int)
```

7. Data Manipulation

Add Column

```python
df["Bonus"] = 100
```

Delete Column

```python
df.drop("Bonus", axis=1)
```

Update Values

```python
df.loc[0,"Age"] = 25
```

Sort Values

```python
df.sort_values("Marks")
```

Sort Index

```python
df.sort_index()
```

Ranking

```python
df["Rank"] = df["Marks"].rank()
```

Map Values

```python
df["Gender"].map({
    "M":1,
    "F":0
})
```

Apply Function

```python
df["Marks"].apply(lambda x:x+5)
```

Apply on Series

```python
df["Age"].map(str)
```

Apply on Entire DataFrame

```python
df.applymap(str)
```

8. Filtering and Querying

Multiple Conditions

```python
df[(df["Age"]>20) &
   (df["Marks"]>80)]
```

Query

```python
df.query("Age > 20")
```

isin()

```python
df[df["City"].isin(["Delhi","Mumbai"])]
```

between()

```python
df[df["Marks"].between(60,90)]
```

9. String Operations

Lower

```python
df["Name"].str.lower()
```

Upper

```python
df["Name"].str.upper()
```

Title

```python
df["Name"].str.title()
```

Strip Spaces

```python
df["Name"].str.strip()
```

Replace

```python
df["Name"].str.replace("a","A")
```

Contains

```python
df["Name"].str.contains("Kr")
```

Split

```python
df["Name"].str.split()
```

Extract

```python
df["Email"].str.extract(r'@(.*)')
```

10. Date and Time Handling

Convert to Date

```python
df["Date"] = pd.to_datetime(df["Date"])
```

Extract Components

```python
df["Date"].dt.year
```

```python
df["Date"].dt.month
```

```python
df["Date"].dt.day
```

```python
df["Date"].dt.weekday
```

Date Filtering

```python
df[df["Date"] > "2025-01-01"]
```

Date Arithmetic

```python
df["Date"] + pd.Timedelta(days=7)
```

11. GroupBy (Most Important Topic)

Grouping

```
Python
df.groupby("Department")
```

Sum

```
Python
df.groupby("Department")["Salary"].sum()
```

Mean

```
Python
df.groupby("Department")["Salary"].mean()
```

Count

```
Python
df.groupby("Department").count()
```

Multiple Aggregations

```
Python
df.groupby("Department")["Salary"].agg(
    ["sum","mean","max","min"]
)
```

12. Statistics

```
Python
df.sum()
```

```
Python
df.mean()
```

```
Python
df.median()
```

```
Python
df.mode()
```

```
Python
```

```
df.std()
```

```
Python
df.var()
```

```
Python
df.corr()
```

```
Python
df.cov()
```

```
Python
df.quantile(0.25)
```

13. Combining DataFrames

Concat

```
Python
pd.concat([df1,df2])
```

Merge

```
Python
pd.merge(df1,df2,on="ID")
```

Inner Join

```
Python
pd.merge(df1,df2,
         how="inner")
```

Left Join

```
Python
pd.merge(df1,df2,
         how="left")
```

Right Join

```
Python
pd.merge(df1,df2,
         how="right")
```

Outer Join

```python
pd.merge(df1,df2,
         how="outer")
```

Join

```python
df1.join(df2)
```

14. Reshaping Data

Pivot

```python
df.pivot(
    index="Date",
    columns="Product",
    values="Sales"
)
```

Pivot Table

```python
df.pivot_table(
    values="Sales",
    index="Region",
    aggfunc="sum"
)
```

Melt

```python
pd.melt(df)
```

Stack

```python
df.stack()
```

Unstack

```python
df.unstack()
```

Crosstab

```python
pd.crosstab(
    df["Gender"],
    df["Department"]
)
```

15. MultiIndex

Create

```python
df.set_index(
    ["Department","Name"]
)
```

Select

```python
df.loc[("IT","Krishna")]
```

Hierarchical Data

Multiple level indexing for complex datasets.

---

16. Categorical Data

Convert

```python
df["Gender"] = df["Gender"].astype("category")
```

Categories

```python
df["Gender"].cat.categories
```

Rename Categories

```python
```

```
df["Gender"].cat.rename_categories(
    ["Male","Female"]
)
```

17. Window Functions

Rolling Window

```
Python
df["Sales"].rolling(3).mean()
```

Expanding Window

```
Python
df["Sales"].expanding().mean()
```

Cumulative Sum

```
Python
df["Sales"].cumsum()
```

Cumulative Max

```
Python
df["Sales"].cummax()
```

18. Time Series Analysis

DateTime Index

```
Python
df.set_index("Date")
```

Resampling

Monthly Sales

```
Python
df.resample("M").sum()
```

Frequency Conversion

```python
df.asfreq("D")
```

Lag Features

```python
df["Lag1"] = df["Sales"].shift(1)
```

Rolling Statistics

```python
df["Sales"].rolling(7).mean()
```

19. Performance Optimization

Vectorized Operations

✅ Fast

```python
df["Salary"] * 1.1
```

❌ Slow

```python
for x in df["Salary"]:
    pass
```

Memory Optimization

```python
df["Gender"] = df["Gender"].astype("category")
```

Efficient Filtering

```python
df.query("Salary > 50000")
```

Avoid Loops

Prefer:

```
Python
apply()
map()
vectorization
```

20. Data Visualization

Line Plot

```
Python
df.plot()
```

Bar Plot

```
Python
df.plot(kind="bar")
```

Histogram

```
Python
df.plot(kind="hist")
```

Scatter Plot

```
Python
df.plot(kind="scatter",
        x="Age",
        y="Salary")
```

Box Plot

```
Python
df.plot(kind="box")
```

Area Plot

```
Python
df.plot(kind="area")
```

21. Exploratory Data Analysis (EDA)

Dataset Overview

```python
df.head()
df.info()
df.describe()
```

Missing Value Analysis

```python
df.isnull().sum()
```

Outlier Detection

```python
df.boxplot()
```

Correlation Analysis

```python
df.corr()
```

Feature Understanding

```python
df["Gender"].value_counts()
```

Summary Statistics

```python
df.describe(include="all")
```

Pandas Topics You Must Master for Data Science Interviews

1. Series & DataFrame
2. Indexing (`loc`, `iloc`)
3. Missing Values
4. GroupBy

5. Merge & Join
6. Pivot Tables
7. String Operations
8. DateTime Handling
9. Apply / Map
10. EDA Workflow
11. Time Series Basics
12. Performance Optimization

These 12 topics cover roughly **80–90% of Pandas usage in real-world Data Science, Data Analysis, Machine Learning, and internships.**